Pre informe desafío II, UdeABook

Informática II Universidad de Antioquia 2026-2

Oscar Trujillo

Medellín, 9 de octubre de 2026

---

1. Introducción

Este documento es el pre informe del segundo desafío del curso. El desafío consiste en construir la capa lógica de una red social, UdeABook, con programación orientada a objetos en C++ y sin apoyarse en la STL ni en librerías externas. Este pre informe recoge lo que he avanzado hasta ahora en el segundo desafío. Todavía no he terminado de programar la red social, así que me centro en analizar el enunciado y explicar cómo pienso organizar las clases. También describo las decisiones que he tomado para evitar que el programa haga demasiadas operaciones, porque la eficiencia es uno de los puntos importantes de la guía. Es posible que algunos detalles cambien cuando empiece a implementar las funcionalidades. Es un documento de trabajo, así que seguramente algunas cosas cambien cuando empiece a implementar.

2. Análisis del problema

Leyendo el enunciado identifiqué cinco cosas que hay que modelar: usuarios, mensajes, álbumes, fotos y las relaciones de amistad (incluyendo las solicitudes que todavía no se han aceptado). Alrededor de ellas giran todas las funcionalidades: iniciar sesión, administrar amigos, enviar y consultar mensajes, administrar álbumes y fotos, dar me gusta y recibir sugerencias. Además, hay dos requisitos que atraviesan todo: la información tiene que quedar en archivos para que no se pierda entre ejecuciones, y cada funcionalidad debe reportar cuántas iteraciones hizo y cuánta memoria usó.

Las restricciones que más pesaron al diseñar fueron tres. No se puede usar nada de la STL, ni siquiera string o vector, así que todas las estructuras tienen que ser propias y manejar su memoria dinámica. El almacenamiento permanente solo sirve para cargar y guardar, no para procesar, es decir que todo tiene que estar en memoria mientras el programa corre. Y la escala: la prueba del video se hace con al menos mil usuarios, y con mil usuarios una decisión descuidada (guardar las amistades como una lista de parejas y recorrerla cada vez, por ejemplo) se nota de inmediato en el conteo de iteraciones. Por eso decidí desde el principio que los usuarios se identifican internamente por un índice, la posición en la que quedaron al cargarse, y el nombre solo se usa para buscarlos: comparar dos enteros es mucho más barato que comparar dos cadenas.

3. Diseño de clases

Las clases quedan en dos niveles. Las de apoyo (figura 1) no saben nada de la red social y servirían para cualquier otro programa: Texto reemplaza a std::string, ListaDinamica<T> reemplaza a vector, Fecha guarda y compara fechas, MatrizBits es una matriz cuadrada donde cada casilla ocupa un solo bit y Medicion lleva la cuenta de las iteraciones. Estas cinco ya las implementé y las probé por separado, porque todo lo demás se construye encima de ellas y prefería tener la base firme antes de seguir.

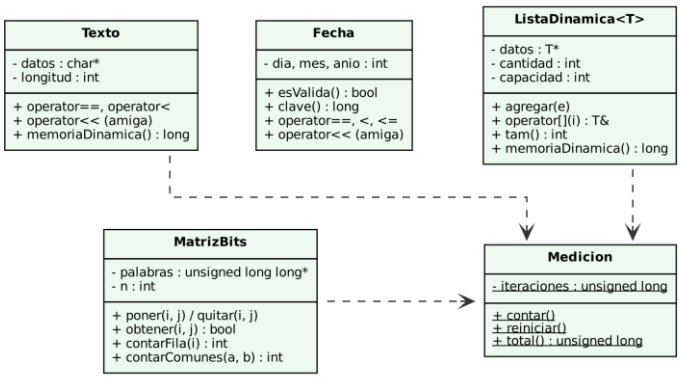


Figura 1. Clases de apoyo. Las del dominio las usan como tipos de sus atributos.

Las del dominio (figura 2) representan el enunciado. Usuario, Mensaje, Album y Foto guardan la información. RedSocial es la dueña de todo: tiene la lista de usuarios, la de todos los mensajes y las dos matrices de bits (amistades y solicitudes pendientes); los usuarios solo guardan punteros a los mensajes que les corresponden, para no tener nada repetido. Archivos se encarga de cargar y guardar, y Menu de la consola, de modo que la lógica queda separada de la interfaz.

En cuanto a lo que pide el enunciado de POO: las clases que manejan memoria dinámica (Texto, ListaDinamica y MatrizBits) tienen constructor de copia, destructor y operador de asignación; Texto y Fecha sobrecargan los operadores de comparación y el operador << como función amiga; ListaDinamica es una plantilla y Medicion usa miembros estáticos para que cualquier clase pueda sumar iteraciones sin pasar un contador de un lado a otro.

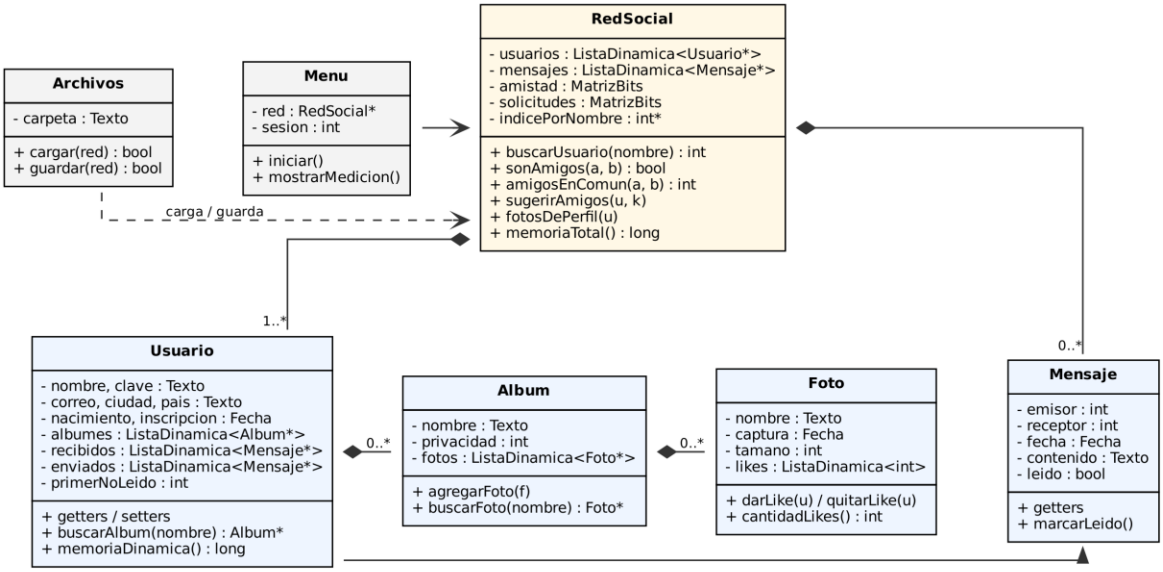


Figura 2. Clases del dominio y sus relaciones.

4. Decisiones pensando en la eficiencia

Amistades y solicitudes

Esta fue la decisión más importante. Mi primera idea fue que cada usuario tuviera una lista de amigos, pero preguntar si A y B son amigos implicaría recorrerla, y calcular los amigos en común implicaría comparar dos listas completas. En lugar de eso uso una matriz de n×n bits donde el bit (i, j) está encendido si i y j son amigos. Elegí representar las amistades con una matriz de bits porque necesito consultar con frecuencia si dos usuarios son amigos. Para mil usuarios, la matriz tendría un millón de posiciones y ocuparía aproximadamente 122 KiB si cada bit se almacena de forma compacta. Además, para buscar amigos en común se pueden comparar las filas de los dos usuarios mediante operaciones AND. Si proceso bloques de 64 bits, serían alrededor de 16 bloques por comparación. Esta es la ventaja que espero obtener con el diseño; el número de iteraciones que registre la implementación me permitirá comprobarlo. Las solicitudes pendientes van en una segunda matriz igual, así que validar que una solicitud no se repita, o que no se le envíe a alguien que ya es amigo, también es inmediato. La idea de trabajar con bits la traje del primer desafío, donde me sirvió para comprimir el tablero.

Búsqueda por nombre

Varias funcionalidades reciben el nombre de un usuario (iniciar sesión, enviar una solicitud, mandar un mensaje). Para buscar usuarios por nombre, mi propuesta es ordenar los índices una sola vez cuando cargue los datos. Utilizaría merge sort para ordenarlos y después una búsqueda binaria para encontrar cada nombre. Con mil usuarios, la búsqueda binaria debería necesitar aproximadamente diez comparaciones en el caso habitual de una búsqueda exitosa, aunque el número exacto dependerá de dónde se encuentre el elemento y de cómo cuente las iteraciones.

Mensajes

Como los mensajes se crean en orden, la lista de recibidos de cada usuario ya queda ordenada por fecha sin tener que ordenarla. Para consultar entre dos fechas hago una búsqueda binaria del primer mensaje con fecha mayor o igual a la inicial y recorro solo hasta pasarme de la final. Para los no leídos aproveché que la opción de leer mensajes nuevos los marca todos de una vez: eso significa que siempre son los últimos de la lista, así que basta con guardar la posición del primero y no hay que revisar la bandera de cada mensaje.

Sugerencias de amistad

Recorro los demás usuarios, descarto los que ya son amigos (un bit) y a los que quedan les calculo los amigos en común con el AND de filas. Como se piden máximo veinte, no ordeno los mil candidatos: mantengo un arreglo pequeño con los mejores encontrados hasta el momento y voy insertando ahí.

Fotos de perfil

Las prioridades 1, 2,5 y 3,5 las manejo como pesos enteros 2, 5 y 7, que conservan la proporción, para hacer el sorteo con aritmética entera. Cuando un amigo sale elegido su peso pasa a cero, para que no se repita dentro del mismo grupo de tres fotos.

Memoria y medición

Las listas duplican su capacidad cuando se llenan, así que agregar al final no cuesta una copia completa cada vez. Las rutas de las fotos no se guardan: se arman en el momento con el nombre del usuario, del álbum y de la foto, lo que ahorra una cadena por foto. Los objetos se pasan por referencia constante para no copiarlos. Para la medición, cada ciclo de mis estructuras llama a Medicion::contar(), de modo que al terminar cada funcionalidad el programa muestra las iteraciones totales; la memoria se calcula sumando el sizeof de cada objeto más lo que reporta cada estructura sobre su memoria dinámica.

5. Almacenamiento

Voy a usar archivos de texto plano, uno por tipo de dato, con los campos separados por una barra vertical y un registro por línea. Lo escogí porque es fácil de generar (necesito un programa aparte que cree los mil usuarios de prueba) y fácil de revisar a mano si algo sale mal. Se cargan completos al iniciar y se escriben al cerrar sesión o salir.

Archivo	Qué guarda cada línea
usuarios.txt	nombre | clave | fecha de nacimiento | correo | ciudad | país | fecha de inscripción. La línea en la que queda cada usuario es su índice.
amistades.txt	índiceA índiceB, una línea por pareja (solo una vez, porque la amistad es simétrica).
solicitudes.txt	índice de quien envía, índice de quien recibe.
mensajes.txt	emisor | receptor | fecha | leído (0 o 1) | contenido. Van en orden cronológico; el contenido va de último porque puede tener cualquier carácter.
albumes.txt	índice del dueño | nombre del álbum | privacidad (0 público, 1 solo amigos, 2 privado).
fotos.txt	índice del dueño | álbum | nombre de la foto | fecha de captura | tamaño | índices de quienes dieron me gusta.

6. Supuestos y dudas

- El enunciado habla de iniciar sesión con credenciales, pero entre los datos del usuario no hay contraseña. Por ahora asumo que se agrega ese campo; si no, la sesión se iniciaría solo con el nombre.
- Para poder quitar un me gusta hay que saber quién lo dio, así que cada foto guarda la lista de índices de los usuarios que lo dieron y no solo un contador.
- Las prioridades de las fotos de perfil dicen 10 o menos, entre 11 y 20, y más de 21, y el 21 queda sin definir. Lo voy a tratar como más de 20.
- Asumo que los mensajes solo llevan fecha y no hora, y que entre mensajes del mismo día manda el orden en que se enviaron.
- Asumo que los mensajes solo se pueden enviar a amigos, porque el enunciado dice "dado el nombre de un amigo".

7. Plan de trabajo

Hasta hoy tengo el repositorio creado, las cinco clases de apoyo implementadas y probadas, y el proyecto compilando en Qt Creator. Para la próxima semana el orden es: primero Mensaje, Foto, Album y Usuario, que son las más sencillas; luego RedSocial con amigos, mensajes y sugerencias, donde está la mayor parte de la lógica; después Archivos junto con un programa que genere los mil usuarios de prueba; y al final Menu, álbumes y fotos de perfil. Pienso hacer un commit cada vez que termine una clase y dejar el informe y el video para los últimos dos días.